

Section 4: Assembling and Deploying Applications (22%) - Study Guide

Disclaimer: Search results confirmed many concepts via direct databricks.com sources. Official doc page URLs were found for several topics (Model Serving, Vector Search, MLflow Unity Catalog registration). All concepts below are grounded in current Databricks products. Verify at docs.databricks.com using the search terms provided at the end of each section.

Terminology Breakdown

- **pyfunc (mlflow.pyfunc):** MLflow's generic "Python function" flavor that lets you wrap any custom Python logic (including pre/post-processing) as a standard, deployable MLflow model.
- **PythonModel:** A base class in `mlflow.pyfunc` that you subclass to create a custom model. The two key methods are `load_context()` (for loading artifacts once at startup) and `predict()` (for inference logic).
- **Model Flavor:** In MLflow, a "flavor" is the framework-specific wrapper that tells MLflow how to save, load, and serve a model (e.g., `mlflow.langchain`, `mlflow.pyfunc`, `mlflow.sklearn`).
- **Model Signature:** A formal schema attached to a logged MLflow model specifying the exact data types and shapes for inputs and outputs. **Required** when registering to Unity Catalog.
- **Input Example:** A sample payload logged alongside an MLflow model. When provided, MLflow 2.5+ automatically infers the model signature from it.
- **Model Registry / Unity Catalog Model Registry:** The Databricks service (backed by Unity Catalog) for versioning, aliasing, and governing ML models. Models are referenced with a three-level namespace: `catalog.schema.model_name`.
- **Model Serving Endpoint:** A managed REST API endpoint in Databricks that hosts an ML model for low-latency, real-time or batch inference. Controlled via Unity Catalog permissions.
- **Service Principal:** A non-human identity (like a machine account) used for automated processes and production workloads, recommended over personal user accounts for endpoint creation.
- **Databricks Secrets:** A secure store for API keys, tokens, and credentials. Referenced in model serving configurations as environment variables so they are never exposed in plain text.
- **Delta Sync Index:** A Mosaic AI Vector Search index type that automatically stays synchronized with a source Delta table via Change Data Feed (CDF).
- **Direct Vector Access Index:** A Vector Search index type where you manage the embeddings yourself (pre-computed), giving full control over the indexing process.

- **CONTINUOUS sync:** A Vector Search pipeline mode that updates the index in near real-time as the source Delta table changes.
 - **TRIGGERED sync:** A Vector Search pipeline mode where the index only updates when you manually trigger a sync.
 - **VectorSearchClient:** The Python SDK client (`databricks.vector_search.client.VectorSearchClient`) used to programmatically create, manage, and query Vector Search indexes.
 - **ai_query()** : A Databricks SQL function that allows you to invoke any model serving endpoint directly within a SQL query, making it ideal for batch inference across entire Delta tables.
 - **Batch Inference:** Running a model against a large, static dataset (as opposed to real-time, one-at-a-time inference). Databricks uses `ai_query()` with serverless compute for this.
 - **MCP (Model Context Protocol):** An open standard for connecting AI agents to external tools and data sources using a standardized client-server interface.
 - **Managed MCP Servers:** Databricks-provided MCP servers offering zero-config access to Databricks-native resources (Unity Catalog, Vector Search, Genie Agents).
 - **External MCP Servers (MCP Services):** Third-party MCP servers (e.g., for GitHub, Slack) that are registered in Unity Catalog and governed via the Unity AI Gateway.
 - **Custom MCP Servers:** User-built MCP servers hosted on Databricks Apps for fully custom tool logic.
 - **MLflow Prompt Registry:** A centralized version control system for prompt templates, with aliases (e.g., `production`, `staging`) that enable environment promotion without code changes.
 - **Databricks Asset Bundles (DABs):** An Infrastructure-as-Code (IaC) framework for Databricks that lets you define, version-control, and deploy resources (jobs, endpoints, vector search indexes) via a `databricks.yml` config file.
 - **Databricks Apps:** A Databricks service for hosting interactive web applications (built with Streamlit, Gradio, Flask, FastAPI) within the Databricks security and governance perimeter.
 - **Lakebase:** A fully managed, serverless Postgres-compatible database on Databricks, used for storing persistent agent state, conversation history, and structured memory.
 - **Short-term Memory (Agent):** Conversation history stored within a single session, typically via thread IDs and checkpointers (e.g., LangGraph's checkpointers).
 - **Long-term Memory (Agent):** Information persisted across multiple sessions in a database (like Lakebase), such as user preferences or accumulated context.
-

1. Code a chain using a pyfunc model with pre- and post-processing

Concept: When a simple LangChain or built-in MLflow flavor does not give you enough control, you use `mlflow.pyfunc` to create a fully custom Python wrapper that can include any arbitrary pre-processing (input validation, formatting) and post-processing (output parsing, filtering) steps around your model call.

Databricks Context & Implementation: You subclass `mlflow.pyfunc.PythonModel` and implement two methods: - `load_context(self, context)` : Runs once when the endpoint starts. Load heavy objects here (LLM client, tokenizer, config). - `predict(self, context, model_input)` : Runs on every request. Chain your pre-processing → model call → post-processing here.

```
import mlflow.pyfunc

class CustomRAGChain(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        # Load config or clients once on startup
        self.llm_client = load_llm_client()

    def _preprocess(self, raw_input):
        # e.g., validate input, extract query field, sanitize
        return raw_input["query"].strip().lower()

    def _postprocess(self, raw_output):
        # e.g., extract only the answer field, add citation format
        return {"answer": raw_output.content, "source": "internal_docs"}

    def predict(self, context, model_input):
        processed = self._preprocess(model_input)
        raw_output = self.llm_client.invoke(processed)
        return self._postprocess(raw_output)

# Log the model
with mlflow.start_run():
    mlflow.pyfunc.log_model(
        artifact_path="rag_chain",
        python_model=CustomRAGChain()
    )
```

Where to verify in Databricks Docs: Search `docs.databricks.com` for: `* "mlflow.pyfunc PythonModel custom" * "Deploy custom Python model Databricks Model Serving"`

2. Control access to resources from model serving endpoints

Concept: When a model serving endpoint is created, it uses the identity of its creator to access Unity Catalog resources. You must configure three types of access control correctly.

Databricks Context & Implementation:

- * **Who can CALL the endpoint (invocation):** Use the **Permissions** tab in the Serving UI to grant `CAN QUERY` to specific users, groups, or Service Principals. For production app-to-app calls, use **OAuth M2M** (Machine-to-Machine) with a Service Principal — never personal access tokens.
- * **Who can ACCESS RESOURCES from inside the endpoint (data access):** The endpoint creator's identity is recorded permanently. That identity must have:
 - * `EXECUTE` privilege on any Unity Catalog Functions the model calls.
 - * `SELECT` on any Delta tables accessed.
 - * `READ FILES` on any UC Volumes used.
- * **Best practice:** Always create production endpoints under a dedicated **Service Principal**, not a personal user account.
- * **External credentials (API keys):** Store third-party API keys (e.g., OpenAI) in **Databricks Secrets**. Reference them as environment variables in the endpoint configuration. The endpoint creator must have `READ` access to those secret scopes.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Model serving endpoint permissions Databricks" * "Databricks Secrets model serving"

3. Code a simple chain according to requirements

Concept: For a straightforward RAG chain — no complex custom logic required — use `mlflow.langchain.log_model()` directly rather than wrapping everything in a pyfunc.

Databricks Context & Implementation: A simple LangChain chain on Databricks:

```

from langchain_community.chat_models import ChatDatabricks
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
import mlflow

# 1. Define components
llm = ChatDatabricks(endpoint="databricks-meta-llama-3-70b-instruct")
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer the question based on the context below."),
    ("human", "Context: {context}\n\nQuestion: {question}")
])
output_parser = StrOutputParser()

# 2. Assemble the chain using the | (pipe) operator
chain = prompt | llm | output_parser

# 3. Log to MLflow
with mlflow.start_run():
    mlflow.langchain.log_model(
        lc_model=chain,
        artifact_path="simple_chain",
        input_example={"context": "...", "question": "..."}
    )

```

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Log and load LangChain models MLflow Databricks"

4. Choose the basic elements needed to create a RAG application: model flavor, embedding model, retriever, dependencies, input examples, model signature

Concept: A RAG application on Databricks has specific required building blocks. You must know what each element does and why it is needed.

Databricks Context & Implementation:

Element	What It Is	Why It's Needed
Model Flavor	<code>mlflow.langchain</code> or <code>mlflow.pyfunc</code>	Tells MLflow how to save/load/serve the chain
Embedding Model	A Foundation Model API endpoint (e.g., <code>databricks-bge-large-en</code>)	Converts text to vectors for Vector Search indexing and querying
Retriever	A <code>DatabricksVectorSearch</code> retriever pointing to your Vector Search index	Fetches the relevant document chunks given a user query
Dependencies	Python libraries (e.g., <code>langchain</code> , <code>databricks-vectorsearch</code>) listed in <code>pip_requirements</code>	Ensures the serving environment can reproduce the chain
Input Example	A sample dict like <code>{"query": "What is our refund policy?"}</code>	Used to auto-infer the model signature; required for Unity Catalog registration
Model Signature	Auto-inferred from input example or manually defined with <code>mlflow.models.infer_signature()</code>	Validates API payloads and is required for Unity Catalog registration

Where to verify in Databricks Docs: Search `docs.databricks.com` for: `* "Build a RAG chatbot Mosaic AI" * "MLflow model signature input example"`

5. Register the model to Unity Catalog using MLflow

Concept: After logging a model in an MLflow run, you register it to the Unity Catalog Model Registry so it can be versioned, governed, and deployed to serving endpoints.

Databricks Context & Implementation:

```

import mlflow

# Point MLflow at Unity Catalog
mlflow.set_registry_uri("databricks-uc")

# Method 1: Register during logging (recommended – one step)
with mlflow.start_run():
    mlflow.langchain.log_model(
        lc_model=chain,
        artifact_path="rag_chain",
        input_example={"query": "What is the return policy?"},
        registered_model_name="main.my_schema.rag_app" # catalog.schema.model
    )

# Method 2: Register a previously logged model
mlflow.register_model(
    model_uri="runs:/<run_id>/rag_chain",
    name="main.my_schema.rag_app"
)

```

Key requirements: * Model name must follow the three-level namespace: `catalog.schema.model_name`.
 * A **model signature is required** by Unity Catalog. Use `input_example` to auto-infer it. * You need `CREATE MODEL` privilege on the target schema.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Register model Unity Catalog MLflow"

6. Create and query a Vector Search index

Concept: A Mosaic AI Vector Search index is the backbone of RAG retrieval. You must know the steps to create it and how to query it.

Databricks Context & Implementation:

Step 1 — Prerequisites:

```

-- Enable Change Data Feed on your source Delta table
ALTER TABLE main.my_schema.chunked_docs
SET TBLPROPERTIES (delta.enableChangeDataFeed = true);

```

Step 2 — Create the index (Python SDK):

```

from databricks.vector_search.client import VectorSearchClient

vsc = VectorSearchClient()

index = vsc.create_delta_sync_index(
    endpoint_name="my_vs_endpoint",
    source_table_name="main.my_schema.chunked_docs",
    index_name="main.my_schema.chunked_docs_index",
    pipeline_type="TRIGGERED",          # or "CONTINUOUS"
    primary_key="id",
    embedding_source_column="chunk_text",
    embedding_model_endpoint_name="databricks-bge-large-en"
)

```

Step 3 — Query the index:

```

results = index.similarity_search(
    query_text="What is the refund policy?",
    columns=["id", "chunk_text"],
    num_results=5
)

```

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Create Mosaic AI Vector Search index" * "Query Vector Search index Databricks"

7. Identify how to serve an LLM application that leverages Foundation Model APIs

Concept: Databricks Foundation Model APIs expose hosted LLMs as REST endpoints. You serve your LLM application by pointing your chain to these endpoints — you do not manage the model infrastructure.

Databricks Context & Implementation: Databricks Foundation Model APIs offer three access modes: *

Pay-per-token: Serverless, per-token billing. Best for prototyping. No capacity reservation needed. *

Provisioned Throughput: Dedicated token-per-second capacity. Required for production SLAs, fine-tuned models, or compliance (HIPAA). You reserve throughput in advance. * **AI Functions:** Optimized for batch processing. Call models directly from SQL using `ai_query()` .

In your application code (LangChain), you reference a Foundation Model API endpoint by name:

```
from langchain_community.chat_models import ChatDatabricks
llm = ChatDatabricks(endpoint="databricks-meta-llama-3-70b-instruct")
```

The endpoint URL, authentication, and scaling are all managed by Databricks.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Databricks Foundation Model APIs" * "Provisioned throughput model serving"

8. Explain the key concepts and components of Mosaic AI Vector Search

Concept: You need a conceptual understanding of how Mosaic AI Vector Search works, not just how to call it.

Databricks Context & Implementation: * **Vector Search Endpoint:** A compute cluster managed by Databricks that serves one or more Vector Search indexes. Created separately before creating an index. * **Index Types:** * **Delta Sync Index:** Auto-syncs from a source Delta table (requires CDF). Best for most RAG use cases. * **Direct Vector Access Index:** You manage uploading pre-computed embeddings yourself. Use when you need full control or your embeddings come from an external source. * **Embedding Source:** * **Databricks-managed embeddings:** Specify an embedding model endpoint. Databricks computes embeddings automatically when data is written to the source Delta table. * **Self-managed embeddings:** You pre-compute and store embeddings in the Delta table yourself. * **Sync Modes:** CONTINUOUS (near real-time) vs. TRIGGERED (manual). * **Governance:** Vector Search indexes are Unity Catalog objects. Access is controlled by standard UC permissions.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Vector Search overview"

9. Identify batch inference workloads and apply `ai_query()` appropriately

Concept: Batch inference means running a model over a large dataset (thousands to millions of rows) rather than one request at a time. `ai_query()` is the Databricks tool for this.

Databricks Context & Implementation: * **When to use batch inference:** Nightly enrichment of a product catalog, scoring all customer support tickets for sentiment, extracting entities from a year's worth of contracts. * **`ai_query()` syntax:**

```
SELECT
  id,
  ai_query(
    'databricks-meta-llama-3-70b-instruct', -- endpoint name
    CONCAT('Classify this review as positive or negative: ', review_text)
  ) AS sentiment
FROM main.my_schema.customer_reviews;
```

- **Requirements:** Serverless SQL Warehouse (not Pro or Classic), Databricks Runtime 18.2+.
- **Optimization tips:** Submit the full dataset in a single query (don't manually loop). Databricks automatically parallelizes across executors and handles rate-limit retries.
- **Monitoring:** Use the Query Profile in Databricks SQL to see completed/failed inference counts and execution time.

Where to verify in Databricks Docs: Search docs.databricks.com for: * "ai_query function Databricks SQL" * "Batch inference AI functions"

10. Configure vector search for a particular solution based on number of embeddings, update frequency, latency, and cost requirements

Concept: Choosing the right Vector Search configuration is an engineering trade-off decision across four dimensions.

Databricks Context & Implementation:

Decision Factor	Configuration Choice	Rationale
High volume of embeddings (millions+)	Scale up Vector Search endpoint size	More compute needed to handle large index and concurrent queries
Frequent data updates (real-time pipeline)	<code>pipeline_type="CONTINUOUS"</code>	Near real-time sync; higher ongoing compute cost
Infrequent updates (daily batch)	<code>pipeline_type="TRIGGERED"</code>	More cost-efficient; sync manually or on a schedule
Low-latency requirement (<100ms)	Use Databricks-managed embeddings + CONTINUOUS sync	Avoids embedding compute lag at query time
Cost-sensitive workload	Self-managed embeddings + TRIGGERED sync	You compute embeddings cheaply in batch; only sync when needed
Compliance / data residency	Databricks-managed endpoint in same region	Data never leaves your cloud region

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Mosaic AI Vector Search configuration" * "Vector Search endpoint sizing"

11. Configure a persistent datastore to store and retrieve intermediate memory or structured information

Concept: Stateful agents need to remember what happened in previous turns (short-term memory) and even across multiple sessions (long-term memory). Databricks provides two options.

Databricks Context & Implementation: * **Option 1 — Managed Agent Memory (Beta):** A fully managed, Unity Catalog-governed service. Zero infrastructure to manage. Ideal for standard per-user, cross-session memory (e.g., remembering user preferences). * **Option 2 — Self-Managed with Lakebase:** Use **Lakebase** (Databricks' managed serverless Postgres-compatible database) as the durable persistence layer. * **Short-term memory (within a session):** Use LangGraph's Checkpointer to persist conversation turns using a thread ID. Each message is written to a Lakebase table row. * **Long-term memory (cross-session):** Use LangGraph's Store API to read/write key insights and user context to Lakebase tables, allowing the agent to recall facts across days or weeks.

Choosing between them:

Need	Use
Simple per-user memory, no custom schema	Managed Agent Memory
Custom schema, complex queries, high concurrency	Lakebase (self-managed)

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Agent memory Databricks" * "Lakebase Databricks Postgres"

12. Apply CI/CD best practices such as updating a Vector Search index, promoting prompts across environments, and testing individual components of an agent

Concept: Production-grade GenAI applications require the same software engineering discipline as traditional apps — version control, automated testing, and environment promotion.

Databricks Context & Implementation: * **Infrastructure as Code:** Use **Databricks Asset Bundles (DABs)** to define Vector Search endpoints, indexes, and serving endpoints in a `databricks.yml` file that is version-controlled in Git. * **Updating a Vector Search index:** In CI/CD, trigger a Vector Search sync via the SDK or REST API as part of your deployment pipeline (or rely on CONTINUOUS sync). Use DABs to ensure the index configuration is consistent across environments. * **Promoting prompts across environments:** Use the **MLflow Prompt Registry** with environment **aliases** (e.g., `staging`, `production`). Your application code loads the prompt by alias, not by version number. Promoting means simply re-assigning the alias: no code change required. * **Testing individual agent components:** Write unit tests (using `pytest`) for each component in isolation: test the retriever independently, test the prompt template, test the output parser. Run integration tests against a staging environment before promoting to production. * **Pipeline:** Git push → `databricks bundle validate` → unit tests → `databricks bundle deploy` to staging → smoke test → promote to production.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Databricks Asset Bundles CI/CD" * "MLflow Prompt Registry aliases"

13. Integrate managed, external, and custom MCP servers based on given application requirements

Concept: MCP (Model Context Protocol) is the open standard for connecting AI agents to external tools. Databricks classifies MCP server deployments into three types, each for different use cases.

Databricks Context & Implementation:

Type	What It Is	When to Use
Managed MCP Servers	Databricks-provided, zero-config servers	Accessing Databricks-native resources: Unity Catalog tables/functions, Vector Search indexes, Genie Agents
External MCP Servers	Third-party MCP servers registered as UC-governed "MCP Services"	Connecting to GitHub, Slack, Jira, or other external services with Databricks governance applied
Custom MCP Servers	User-built MCP servers hosted on Databricks Apps	Specialized tool logic that doesn't fit a managed or external server

- **Governance:** All MCP server traffic routes through the Unity AI Gateway, providing centralized auditability and security.
- **Connection example (Genie as MCP tool):** An agent connects to a Genie Agent via its managed MCP URL: `https://<workspace>/api/2.0/mcp/genie/{genie_space_id}`

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MCP servers Databricks" * "Unity Catalog MCP services"

14. Apply prompt version control and manage prompt lifecycle

Concept: Prompts are critical application code. They must be versioned, tested, and promoted across environments just like source code — not hardcoded as string literals in notebooks.

Databricks Context & Implementation: The **MLflow Prompt Registry** provides this capability: *

Versioning: Every change to a prompt creates an immutable, auto-numbered version snapshot. You can see a diff between versions in the MLflow UI. * **Commit messages:** Document *why* a prompt was changed, not just what changed. * **Aliases:** Mutable labels (e.g., `dev`, `staging`, `production`) assigned to specific versions. Your application code loads the prompt by alias — so "promoting" a prompt to production is just re-assigning the `production` alias to a new version. * **Integration with experiments:** Link prompt versions to MLflow runs. When a production incident is traced back to a bad prompt, you can identify exactly which prompt version was active using `mlflow.set_active_model()`. * **Loading a prompt in code:**

```
import mlflow

prompt = mlflow.load_prompt("prompts:/my_rag_prompt/production")
```

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "MLflow Prompt Registry Databricks"

15. Develop an appropriate interactive user-facing interface for an agent usage scenario (Apps, Slack, Teams, etc.)

Concept: After building and deploying an agent, end users need a way to interact with it. The right interface depends on where users live and how they prefer to work.

Databricks Context & Implementation:

User Scenario	Recommended Interface	How
Internal business users who live in Slack	Databricks Genie app for Slack	Admin enables preview; users mention <code>@Genie</code> in channels. Pin specific Genie Agents to channels.
Internal users who live in Microsoft Teams	Databricks Genie app for Teams	Admin installs from Microsoft marketplace; users mention <code>@Databricks Genie</code> .
Custom agent with a web UI (internal tool)	Databricks Apps (Streamlit / Gradio / FastAPI)	Host a Streamlit or Gradio front-end on Databricks Apps. Built-in SSO and governance. App calls the agent's Model Serving endpoint.
External customer-facing app	Custom web application	External app calls the Databricks Model Serving REST endpoint via OAuth M2M.
Developers / API consumers	Direct REST API	Query the Model Serving endpoint directly using the Databricks SDK or HTTP client.

Databricks Apps key advantage: Applications run inside the Databricks security perimeter — built-in SSO, Unity Catalog identity, no separate deployment infrastructure needed.

Where to verify in Databricks Docs: Search `docs.databricks.com` for: * "Databricks Apps deploy agent" * "Databricks Genie app Slack Teams"